

Verifpal analysis of 01-signature-bypass.vp

August 31, 2026

Abstract

This report describes the analysis of the Verifpal model **01-signature-bypass.vp** against an active attacker, with each principal running 2 concurrent sessions. Of its 2 queries, 2 were broken. Every attack reported here is a witness against the model as written. A query reported as holding means that this search found no attack at these parameters, which is not a proof that none exists.

1 01-signature-bypass.vp

Analysed against an active attacker at 2 sessions per principal, in 170 ms. The result code is **a1c1**.

1.1 Protocol

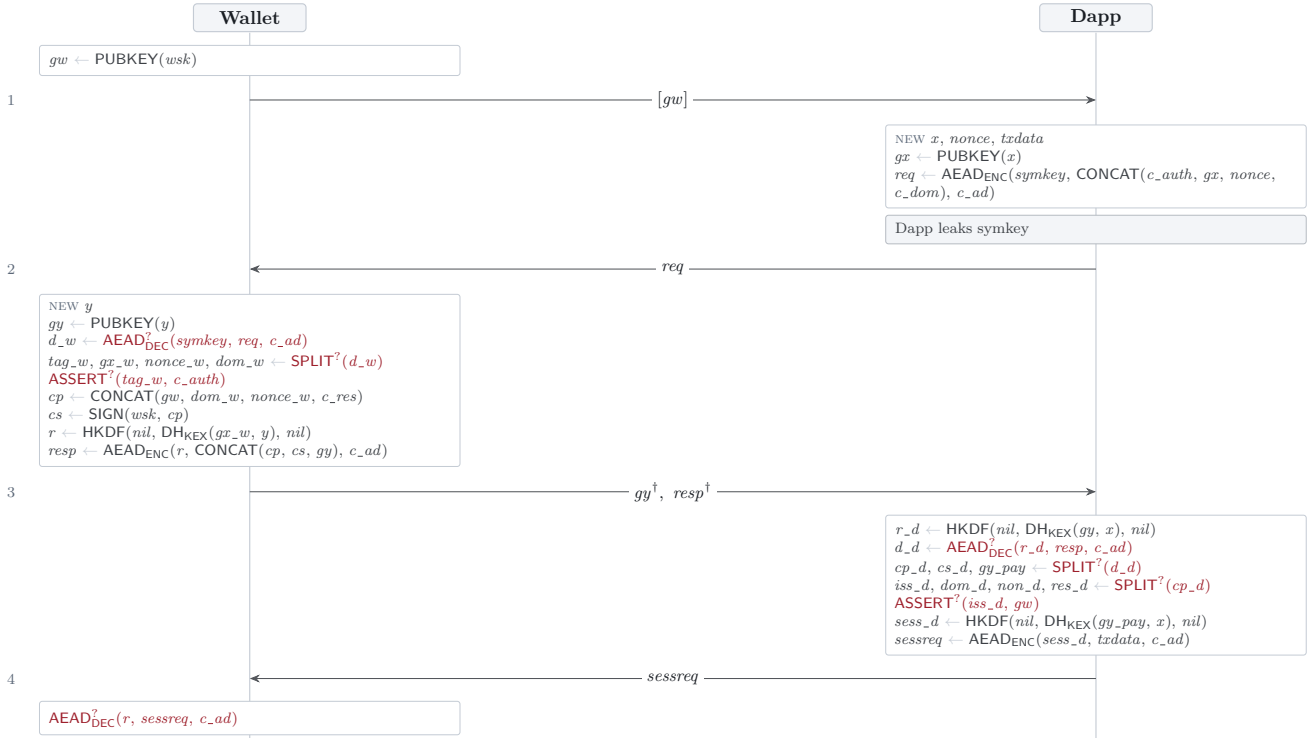


Figure 1: The protocol as written. Guarded values are bracketed; a dagger marks every value an attacker below substitutes or replays.

1.2 Verdicts

Table 1: Every query in the model, with the verdict this run reached.

Query	Verdict	Scope of the verdict	Line
authentication? Wallet $\rightarrow$ Dapp : <i>resp</i>	attack	a witness, shown below	182
confidentiality? <i>txdata</i>	attack	a witness, shown below	184

1.3 Attacks

1.3.1 authentication? **Wallet** $\rightarrow$ **Dapp**: *resp*

resp ($\text{AEAD}_{\text{ENC}}(r_d, d_d, c_ad)$), sent by Attacker and not by Wallet, is successfully used in $\text{AEAD}_{\text{DEC}}^?(r_d, resp, c_ad)$ within Dapp's state.

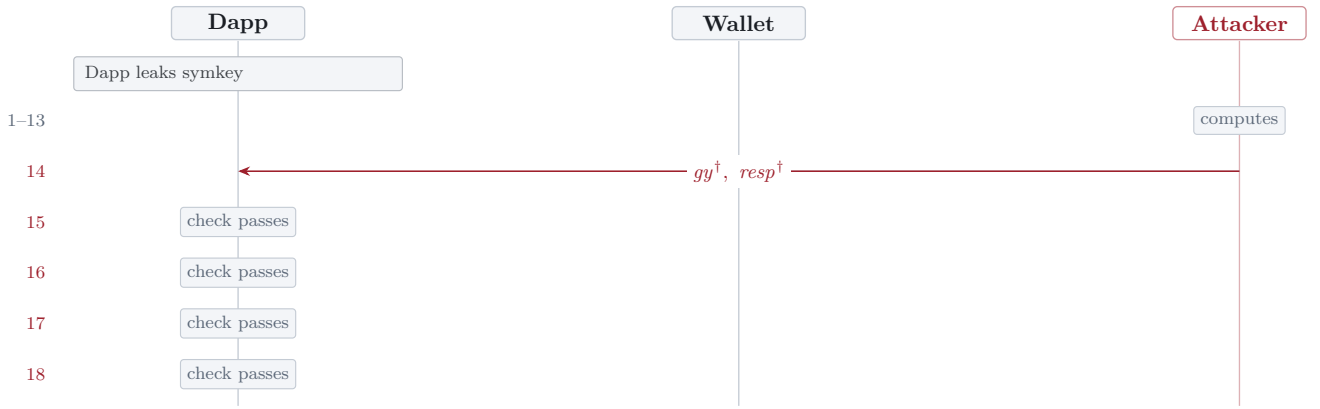


Figure 2: How the attacker breaks authentication? **Wallet** $\rightarrow$ **Dapp**: *resp*. Substituted values carry a dagger.

- ① During Wallet's run (phase 0), attacker constructs $\text{PUBKEY}(\text{nil})$.
- ② During Wallet's run (phase 0), attacker observes *req* on the wire.
- ③ During Wallet's run (phase 0), attacker is handed *symkey* by a leaks declaration.
- ④ During Wallet's run (phase 0), attacker opens *req* with *symkey*, obtaining *d_w*.
- ⑤ During Wallet's run (phase 0), attacker splits *d_w* and takes *gx*.
- ⑥ Attacker constructs $\text{DH}_{\text{KEX}}(\text{gx}, \text{nil})$.
- ⑦ Attacker constructs $\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gx}, \text{nil}), \text{nil})$.
- ⑧ During Wallet's run (phase 0), attacker observes $\text{AEAD}_{\text{ENC}}(\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gy_pay}, \text{nil}), \text{nil}), d_d, c_ad)$ on the wire.
- ⑨ During Wallet's run (phase 0), attacker observes $\text{PUBKEY}(y)$ on the wire.
- ⑩ During Wallet's run (phase 0), attacker constructs $\text{DH}_{\text{KEX}}(\text{gy_pay}, \text{nil})$.
- ⑪ During Wallet's run (phase 0), attacker constructs $\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gy_pay}, \text{nil}), \text{nil})$.
- ⑫ During Wallet's run (phase 0), attacker opens $\text{AEAD}_{\text{ENC}}(\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gy_pay}, \text{nil}), \text{nil}), d_d, c_ad)$ with $\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gy_pay}, \text{nil}), \text{nil})$, obtaining $\text{CONCAT}(cp, cs, \text{gy_pay})$.
- ⑬ Attacker constructs $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_ad)$.
- ⑭ Attacker replaces *gy*, *resp* (sent by Wallet to Dapp) with $\text{PUBKEY}(\text{nil})$, $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_ad)$. (*gy* was $\text{PUBKEY}(y)$; *resp* was $\text{AEAD}_{\text{ENC}}(r, \text{CONCAT}(cp, cs, \text{gy}), c_ad)$) Wallet $\rightarrow$ Dapp
- ⑮ Dapp's $\text{AEAD}_{\text{DEC}}^?(r_d, \text{AEAD}_{\text{ENC}}(r_d, \text{CONCAT}(cp, cs, \text{gy_pay}), c_ad), c_ad)$ passes — the attacker controls one of its inputs.
- ⑯ Dapp's $\text{SPLIT}^?(d_d)$ passes — the attacker controls one of its inputs.
- ⑰ Dapp's $\text{SPLIT}^?(cp_d)$ passes — the attacker controls one of its inputs.
- ⑱ Dapp's $\text{ASSERT}^?(iss_d, gw)$ passes — the attacker controls one of its inputs.

1.3.2 confidentiality? *txdata*

txdata (*txdata*) is obtained by Attacker.

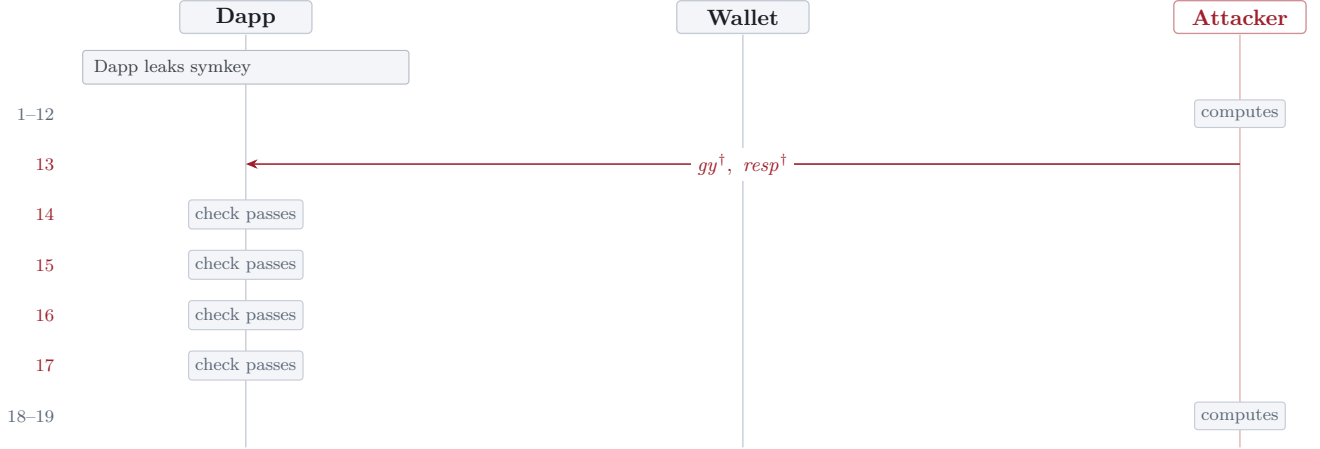


Figure 3: How the attacker breaks confidentiality? *txdata*. Substituted values carry a dagger.

- ① Attacker constructs $\text{PUBKEY}(\text{nil})$.
- ② Attacker observes *req* on the wire.
- ③ Attacker is handed *symkey* by a leaks declaration.
- ④ Attacker opens *req* with *symkey*, obtaining *d_w*.
- ⑤ Attacker splits *d_w* and takes *gx*.
- ⑥ Attacker constructs $\text{DH}_{\text{KEX}}(\text{gx}, \text{nil})$.
- ⑦ Attacker constructs $\text{HKDF}(\text{nil}, \text{DH}_{\text{KEX}}(\text{gx}, \text{nil}), \text{nil})$.
- ⑧ Attacker observes *gw* on the wire.
- ⑨ Attacker splits *d_w* and takes *nonce*.
- ⑩ Attacker constructs *cp*.
- ⑪ Attacker constructs $\text{CONCAT}(\text{cp}, \text{nil}, \text{PUBKEY}(\text{nil}))$.
- ⑫ Attacker constructs $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_ad)$.
- ⑬ Attacker replaces *gy*, *resp* (sent by Wallet to Dapp) with $\text{PUBKEY}(\text{nil})$, $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_ad)$. (*gy* was $\text{PUBKEY}(y)$; *resp* was $\text{AEAD}_{\text{ENC}}(r, \text{CONCAT}(\text{cp}, \text{cs}, \text{gy}), c_ad)$)
Wallet → Dapp
- ⑭ Dapp's $\text{AEAD}_{\text{DEC}}^?(r_d, \text{AEAD}_{\text{ENC}}(r_d, \text{CONCAT}(\text{cp}, \text{nil}, \text{PUBKEY}(\text{nil})), c_ad), c_ad)$ passes — the attacker controls one of its inputs.
- ⑮ Dapp's $\text{SPLIT}^?(d_d)$ passes — the attacker controls one of its inputs.
- ⑯ Dapp's $\text{SPLIT}^?(cp_d)$ passes — the attacker controls one of its inputs.
- ⑰ Dapp's $\text{ASSERT}^?(iss_d, gw)$ passes — the attacker controls one of its inputs.
- ⑱ Attacker observes *sessreq* on the wire.
- ⑲ Attacker opens *sessreq* with *r_d*, obtaining *txdata*.

1.4 Scope and assumptions

Every verdict above was reached against an active attacker, with each principal running 2 concurrent sessions, over exactly the model as written. An attack is a witness and stands on its own. A query reported as holding says only that this search found no attack at those parameters: the search space this engine defines was explored, which is never the space of all attacks.

A Model source

A.1 01-signature-bypass.vp

```

1 // FINDING 1 -a CACAO whose signature does not verify still establishes a
2 // persisted, subscribed session, for an account the attacker names.
3 //
4 // Expected: a1c1 at one session and at two.
5 //
6 // -----
7 // SOURCE OF EVERY MODELLING DECISION BELOW
8 // walletconnect-monorepo @ master
9 // -----
10 //
11 // utils/src/crypto.ts
12 // deriveSymKey(priv, pub) = hkdf(sha256, x25519(priv, pub), undefined,
13 // undefined, 32) -- no salt, no info
14 // hashKey(k) = sha256(k)
15 // type 1 envelope = type | senderPublicKey | iv | sealed
16 // encrypt() = chacha20poly1305(symKey, iv).encrypt(msg)
17 // with NO associated data, so senderPublicKey is
18 // cleartext and unauthenticated.
19 //
20 // core/src/controllers/crypto.ts
21 // encode(): for a type 1 envelope the symKey is NOT the topic's key --
22 // topic = await this.generateSharedKey(senderPublicKey, receiverPublicKey)
23 // so the response is sealed under the very key that becomes the session key.
24 // decode(): the receiver derives that key from the senderPublicKey carried
25 // IN the envelope:
26 // generateSharedKey(params.receiverPublicKey, params.senderPublicKey)
27 //
28 // sign-client/src/controllers/engine.ts authenticate() [dapp side]
29 // publicKey = fresh x25519 keypair, stored under AUTH_PUBLIC_KEY_NAME
30 // responseTopic = hashKey(publicKey) -- hash of a PUBLIC key
31 // request = { authPayload: { type, chains, statement, aud, domain,
32 // version, nonce, iat, exp, nbf, resources },
33 // requester: { publicKey, metadata }, expiryTimestamp }
34 // sent on the pairing topic, sealed under the pairing symKey.
35 //
36 // sign-client/src/controllers/engine.ts approveSessionAuthenticate() [wallet]
37 // receiverPublicKey = pendingRequest.requester.publicKey (= gx here)
38 // senderPublicKey = fresh keypair (= gy here)
39 // responseTopic = hashKey(receiverPublicKey)
40 // sessionTopic = generateSharedKey(senderPublicKey, receiverPublicKey)
41 // result = { cacaos: auths,
42 // responder: { publicKey: senderPublicKey, metadata } }
43 //
44 // NOTE the responder public key appears TWICE and the two are never compared:
45 // once in the envelope (which decides the decryption key) and once inside the
46 // ciphertext (which decides the session topic). They are modelled separately,
47 // as gy_env and gy_pay.
48 //
49 // utils/src/cacao.ts
50 // validateSignedCacao({ cacao }):
51 // reconstructed = formatMessage(payload, payload.iss) // payload IS cacao.p
52 // return verifySignature(getDidAddress(payload.iss), reconstructed, ...)
53 //
54 // The signed statement is therefore a deterministic function of the CACAO's
55 // own payload, and the check is purely internal consistency: "the address
56 // named in `iss` signed this text". It is NOT compared against the request.
57 // Grepping the engine confirms it: `nonce` occurs only where a request is
58 // built and in isValidAuthenticate's input validation, never in a comparison
59 // against a returned CACAO, and neither `domain` nor `aud` is ever compared.
60 // The model reflects that by signing the payload directly.

```

```

61 //
62 // -----
63 // THE BEHAVIOUR UNDER TEST
64 // -----
65 //
66 // engine.ts, dapp-side onAuthenticate:
67 //
68 // const isValid = await validateSignedCacao({ cacao, projectId });
69 // if (!isValid) {
70 //   this.client.logger.error(cacao, "Signature verification failed");
71 //   reject(getSdkError("SESSION_SETTLEMENT_FAILED", ...));
72 // }
73 // ... falls through ...
74 // approvedChains = [getNamespacedDidChainId(payload.iss)]
75 // parsedAddress = getDidAddress(payload.iss)
76 // approvedMethods = getMethodsFromRecap(recap) // from resources
77 // const sessionTopic = await generateSharedKey(publicKey, responder.publicKey);
78 // await this.client.core.relayer.subscribe(sessionTopic, { transportType });
79 // await this.client.session.set(sessionTopic, session);
80 //
81 // reject() settles the promise but does not return, and onAuthenticate has no
82 // enclosing try/catch, so a CACAO that FAILS verification still reaches session
83 // construction. The approved account and methods are read from that same
84 // unvalidated payload. Store.set persists, so the session survives a restart.
85 // The wallet side of the identical check, ~190 lines below, does
86 // `await this.sendError(...)` and then `throw`.
87 //
88 // A check that does not halt is a SIGNVERIF without `?`. That is the whole
89 // difference between this model and 03-fix-verification.vp.
90 //
91 // -----
92 // PRECONDITION
93 // -----
94 //
95 // To address the response the attacker needs gx, which travels in the request
96 // under the pairing symKey. `leaks symkey` models the pairing URI being
97 // observed -- it is rendered as a QR code for the user to photograph, so it is
98 // exposed to anything that can see the screen. Over link mode this precondition
99 // disappears entirely: the request is a cleartext type 2 envelope in a URL.
100
101 attacker[active]
102
103 principal Wallet[
104   knows private wsk // the user's account key
105   gw = PUBKEY(wsk) // the did:pkh address, public knowledge
106 ]
107
108 Wallet → Dapp: [gw]
109
110 principal Dapp[
111   knows private symkey // pairing symKey, from the URI
112   knows public c_ad
113   knows public c_auth // the wc_sessionAuthenticate method name
114   knows public c_dom // authPayload.domain and .aud
115   generates x
116   generates nonce // authPayload.nonce
117   generates txdata // what the dapp sends once it believes it is authenticated
118   gx = PUBKEY(x) // requester.publicKey; responseTopic = sha256(gx)
119   req = AEAD_ENC(symkey, CONCAT(c_auth, gx, nonce, c_dom), c_ad)
120   leaks symkey

```

```

121 ]
122
123 Dapp → Wallet: req
124
125 principal Wallet[
126     knows private symkey
127     knows public c_ad
128     knows public c_auth
129     knows public c_dom
130     knows public c_res // authPayload.resources, carrying the recap
131     generates y
132     gy = PUBKEY(y)
133     d_w = AEAD_DEC(symkey, req, c_ad)?
134     tag_w, gx_w, nonce_w, dom_w = SPLIT(d_w)?
135     _ = ASSERT(tag_w, c_auth)?
136     // cacao.p: iss, domain, nonce, resources. formatMessage() is a function of
137     // exactly this, so signing it directly is faithful.
138     cp = CONCAT(gw, dom_w, nonce_w, c_res)
139     cs = SIGN(wsk, cp)
140     // The response is sealed under generateSharedKey(gy, gx) -- the session key.
141     r = HKDF(nil, DH_KEX(gx_w, y), nil)
142     // result = { cacaos: [{p, s}], responder: { publicKey } }
143     resp = AEAD_ENC(r, CONCAT(cp, cs, gy), c_ad)
144 ]
145
146 // Type 1 envelope: the senderPublicKey field travels in the clear.
147 Wallet → Dapp: gy, resp
148
149 principal Dapp[
150     // decode(): the decryption key comes from the envelope's senderPublicKey.
151     r_d = HKDF(nil, DH_KEX(gy, x), nil)
152     d_d = AEAD_DEC(r_d, resp, c_ad)?
153     cp_d, cs_d, gy_pay = SPLIT(d_d)?
154     // validateSignedCacao: reconstruct from the payload, verify against the
155     // address the payload itself names. No comparison with nonce or c_dom.
156     iss_d, dom_d, non_d, res_d = SPLIT(cp_d)?
157     // The dapp is authenticating a PARTICULAR account -- it is re-authenticating
158     // a known user, or matching the address against its own records. Without
159     // this the queries below fail for an uninteresting reason: anyone may
160     // answer an authenticate request with a CACAO for their OWN account, and
161     // will of course hold the resulting session. That is connecting a wallet,
162     // not impersonating one. This assertion is what makes the failure mean
163     // "the dapp accepted a session for gw".
164     _ = ASSERT(iss_d, gw)?
165     // AS IMPLEMENTED: verification failure logs and rejects, then falls through.
166     // No `?`, so the dapp does not halt on a bad signature.
167     _ = SIGNVERIF(iss_d, cp_d, cs_d)
168     // sessionTopic = generateSharedKey(publicKey, responder.publicKey) -- note
169     // this uses the key from INSIDE the payload, not the envelope's.
170     sess_d = HKDF(nil, DH_KEX(gy_pay, x), nil)
171     sessreq = AEAD_ENC(sess_d, txdata, c_ad)
172 ]
173
174 Dapp → Wallet: sessreq
175
176 principal Wallet[
177     _ = AEAD_DEC(r, sessreq, c_ad)?
178 ]
179
180 queries[

```

```
181 |      // Can the dapp be made to accept a response the wallet did not author?  
182 |      authentication? Wallet → Dapp: resp  
183 |      // Does the attacker end up holding the session it establishes?  
184 |      confidentiality? txdata  
185 | ]
```

B On soundness

Verifpal is sound but incomplete. Every attack shown here is a genuine attack on the model as written; a query reported as holding means no attack was found within the search this run performed, which is never a proof that none exists.